

Practical (OSSP)

SESSION 5

TASK 1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficient.

```
GNU nano 7.2 session5_task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <time.h>

int main() {
    int pipefd[2];
    pid_t pid;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(1);
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid > 0) {
        // Parent - Producer
        close(pipefd[0]);

        const char *data =
            "Producer sends data through anonymous pipe.\n"
            "This demonstrates producer-consumer communication.\n";

        clock_t start = clock();

        write(pipefd[1], data, strlen(data));

        clock_t end = clock();

        close(pipefd[1]);

        double time_taken =
            ((double)(end - start)) / CLOCKS_PER_SEC;
    }
}
```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^/ Go To Line

```

    printf("Producer: Data sent successfully.\n");
    printf("Communication time: %.6f seconds\n", time_taken);

    wait(NULL);
    printf("Producer: Child process completed.\n");
}
else {
    // Child - Consumer
    close(pipefd[1]);

    char buffer[200];
    int bytes_read;

    bytes_read = read(pipefd[0], buffer, sizeof(buffer) - 1);

    if (bytes_read > 0) {
        buffer[bytes_read] = '\0';

        printf("Consumer: Data received:\n");
        printf("%s", buffer);
        printf("Consumer: Data processed successfully.\n");
    }

    close(pipefd[0]);
    exit(0);
}

return 0;
}

```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line

```

root@HP: ~/shellforge
root@HP:~# cd ~/shellforge
root@HP:~/shellforge# pwd
/root/shellforge
root@HP:~/shellforge# nano session5_task1.c
root@HP:~/shellforge# gcc session5_task1.c -o session5_task1
root@HP:~/shellforge# ./session5_task1
Producer: Data sent successfully.
Consumer: Data received:
Producer sends data through anonymous pipe.
This demonstrates producer-consumer communication.
Communication time: 0.000013 seconds
Consumer: Data processed successfully.
Producer: Child process completed.
root@HP:~/shellforge#

```

TASK 2: Develop a program that executes the equivalent of the shell command: `ls -l grep` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
GNU nano 7.2 session5_task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(1);
    }

    // First child: ls -l
    pid1 = fork();

    if (pid1 < 0) {
        perror("fork");
        exit(1);
    }

    if (pid1 == 0) {
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    // Second child: grep
    pid2 = fork();
```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

```

// Second child: grep
pid2 = fork();

if (pid2 < 0) {
    perror("fork");
    exit(1);
}

if (pid2 == 0) {
    close(pipefd[1]);

    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);

    execlp("grep", "grep", "session5", NULL);

    perror("execlp grep");
    exit(1);
}

// Parent
close(pipefd[0]);
close(pipefd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Pipeline execution completed.\n");

return 0;
}

```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

```

root@HP:~/shellforge# nano session5_task2.c
root@HP:~/shellforge# gcc session5_task2.c -o session5_task2
root@HP:~/shellforge# ./session5_task2
-rwxr-xr-x 1 root root 16480 Aug 27 10:09 session5_task1
-rw-r--r-- 1 root root 1469 Aug 27 10:08 session5_task1.c
-rwxr-xr-x 1 root root 16360 Aug 27 10:10 session5_task2
-rw-r--r-- 1 root root 1052 Aug 27 10:10 session5_task2.c
Pipeline execution completed.
root@HP:~/shellforge#

```